

Section A — Short Answer Questions (Conceptual)

Self-Supervised Learning (SSL)

1. Define SSL and how it differs from supervised and unsupervised learning.

Self-supervised learning constructs *supervision from the data itself* by defining *pretext tasks* whose labels are obtained without human annotation (e.g., predicting parts of the input from other parts). It differs from:

- **Supervised learning:** requires manually labeled input–output pairs and directly optimizes the downstream objective.
- **Unsupervised learning:** aims to model data distribution or structure (e.g., clustering, density estimation) without designing predictive pseudo-labels; SSL uses predictive tasks (pseudo-labels) to learn representations useful for downstream supervised tasks.

2. Three pretext tasks in computer vision and what they force the model to learn.

- **Predict image rotation (e.g., 0°, 90°, 180°, 270°):** forces the network to learn global object orientation and semantic structure so it can discriminate plausible object rotations.
- **Contrastive instance discrimination (SimCLR style):** create two augmented views of the same image; force representations of the same instance to be close and different instances far. Learns invariances to augmentation and instance-level discriminative features.
- **Jigsaw puzzle / patch ordering:** cut an image into patches and predict the correct spatial ordering. Forces learning of spatial relationships and object part configuration.

3. Pretext task vs. downstream task — why not use pretext directly?

Pretext tasks are **proxy objectives** chosen because they are cheaply labeled; they encourage learning useful features but are not the actual task of interest. They may induce useful invariances or structures but do not necessarily align perfectly with downstream metrics (e.g., rotation prediction may not separate fine-grained object classes). Therefore we usually *pretrain* on the pretext task to learn representations, then *fine-tune* or train a small head for the downstream task.

4. Contrastive vs. generative SSL (plus one example each).

- **Contrastive approaches** learn representations by pulling together positive pairs (augmented views of same instance) and pushing apart negatives (other instances).

Example: SimCLR, MoCo. Strength: very strong discriminative features. Weakness: needs careful negative sampling or large batch sizes (or memory banks).

- **Generative approaches** learn by reconstructing parts of input or modeling input distribution (often using reconstruction or likelihood objectives). Example: Denoising autoencoder, Masked Image Modeling (MAE). Strength: can capture structure without negatives; weak: may learn trivial reconstructions if not constrained.
-

Multi-Task Learning (MTL)

5. Define MTL and explain shared layers vs. task-specific heads.

MTL trains a model on several tasks simultaneously to exploit shared structure. Typical architecture: a *shared backbone* (layers h_{θ_0}) computes a common representation; *task-specific heads* f_{θ_t} take that representation and produce task outputs. Shared layers capture common features; heads specialize.

6. Two regularization techniques to prevent negative transfer.

- **Soft parameter sharing / similarity regularizer**: penalize $\|\theta_t - \theta_u\|^2$ weighted by task similarity to discourage unrelated tasks from converging to same parameters.
- **Gradient surgery / gradient projection (e.g., PCGrad) or GradNorm**: manipulate gradients so conflicting gradient directions are corrected (project conflicting gradients or balance magnitudes), reducing interference.

7. Hard vs. soft parameter sharing.

- **Hard sharing**: tasks share the same parameters (backbone) exactly; only heads differ. Strong regularization, efficient but can cause negative transfer.
- **Soft sharing**: each task has separate parameters but regularizers or constraints encourage them to be similar (e.g., L2 penalty between parameters). More flexible; can be tuned to degree of sharing.

8. What is task interference, and mitigation strategies.

Task interference = when gradients for different tasks conflict (opposite directions) causing degraded performance for one or more tasks. Mitigations: gradient projection methods (PCGrad), loss weighting (dynamic task weighting like GradNorm), task grouping/decoupling layers, selective sharing (share only lower layers), or orthogonal parameterization.

Meta-Learning

9. Define meta-learning and the N-way K-shot setup.

Meta-learning trains a learner that can *rapidly adapt* to new tasks from few examples. **N-way K-shot**: each episode is a classification task with N classes and K labeled support examples per class (used for adaptation), plus query examples for evaluation.

10. Compare model-based, metric-based, optimization-based meta-learning.

- **Model-based**: design architectures that store fast adaptation in parameters/state (e.g., memory-augmented networks, conditional networks). Fast inference but often less general.
- **Metric-based**: learn an embedding and classification by similarity (e.g., Prototypical Networks, Matching Networks). Very sample-efficient, simple adaptation (nearest prototype).
- **Optimization-based**: learn an initialization or optimizer so that a few gradient steps produce good task parameters (e.g., MAML, learned optimizers). More general but requires backprop through updates.

11. How MAML works (two-phase training).

- **Inner loop (adaptation)**: for each task, starting from meta-init ϕ , take one or more gradient steps on the task's support set to get task parameters θ_t . Example one step: $\theta_t = \phi - \alpha \nabla_{\phi} L_{\text{support}}(f_{\phi})$.
- **Outer loop (meta-update)**: evaluate θ_t on the task's query set and update ϕ to minimize post-adaptation loss; gradients flow through the inner update(s). This trains ϕ to be a good initialization for quick adaptation.

12. Meta-training vs. meta-testing.

- **Meta-training**: optimize meta-parameters (initialization, metric, or model) across many training tasks; involves inner and outer loops.
- **Meta-testing (deployment)**: given an unseen task, start from learned meta-parameters and perform adaptation steps using the small support set; no meta-gradient updates are computed (we only adapt for that single task).

Parameter-Efficient Fine-Tuning (PEFT)

13. Define PEFT and why essential for LLM adaptation.

PEFT adapts large pre-trained models by training a *small set of additional*

parameters (adapters, prompts, low-rank updates, bias terms) while keeping the base model frozen. Essential because LLMs are huge (memory/storage), so full fine-tuning is costly for many tasks and wastes storage (one full copy per task). PEFT enables low-cost task adaptation and easy deployment.

14. Compare LoRA with adapter methods (architecture & efficiency).

- **LoRA:** freezes base W , learns low-rank factors A, B such that $W' = W + BA$. Very parameter-efficient (parameters scale with rank r) and easy to inject into linear projections (attention/FFN).
- **Adapters:** small bottleneck layers inserted between existing layers (e.g., down-project $\rightarrow$ nonlinearity $\rightarrow$ up-project). Also parameter-efficient but add extra forward computation and latency; LoRA often has lower parameter count and minimal compute overhead (simple matmul addition).

15. Low-rank assumption behind LoRA and why it's reasonable.

Assumption: task-specific changes to large weight matrices lie in a low-dimensional subspace; therefore a low-rank update captures most of useful adaptation. Reasonable because empirical observations show fine-tuning updates have low-rank structure (redundancy in parameter updates) and many adaptation directions are correlated.

16. Advantages of PEFT over full fine-tuning for deployment.

- Much smaller per-task storage (store small adapters vs. whole model).
- Lower GPU memory during training (fewer trainable params).
- Faster iteration and safer: base model remains unchanged.
- Easy to switch tasks by loading lightweight adapters.
- Less risk of catastrophic forgetting or corrupting base model.

Section B — Problem-Solving Questions

Self-Supervised Learning Problems

17. Design Task (unlabeled audio):

Design: **Contrastive predictive coding (CPC)** style pretext on audio segments.

Procedure:

- Split audio into contiguous segments (e.g., frames or short windows).
- Create augmentations (time-masking, frequency masking, small pitch/time shifts).

- Build encoder g_ϕ that maps a short audio chunk to an embedding; build an autoregressive aggregator c_ψ over previous embeddings.
- Pretext: predict future segment embeddings (or their quantized identifiers) from context using contrastive loss (InfoNCE) with negative samples from other audio segments or other recordings.

Justification: learns temporal structure, phonetic and speaker-invariant features, robust to augmentations. CPC encourages representations predictive of future context — useful for speech recognition, speaker ID, downstream classification.

18. Analysis: rotation pretext fails downstream — two possible reasons & fixes.

- **Pretext misalignment:** rotation prediction focuses on coarse global cues and orientation, not class-discriminative texture/shape needed for classification. *Fix:* choose richer pretext (contrastive or patch-based) or combine multiple pretexts.
 - **Insufficient augmentations or capacity:** task may be solved using trivial cues (edges, borders) leading to shortcuts. *Fix:* stronger augmentation, cropping, remove trivial cues, or increase model capacity and long context.
-

Multi-Task Learning Problems

19. Architecture design for detection, segmentation, depth.

- **Shared backbone:** low- and mid-level conv blocks (feature pyramid) — captures edges, textures, multi-scale features.
- **Task-specific heads:**
 - **Detection head:** region proposal / anchor head + box regression and class logits (uses FPN features + specialized convs).
 - **Segmentation head:** upsampling / decoder (e.g., upsample + skip connections) producing per-pixel class map.
 - **Depth head:** regression decoder (similar to segmentation decoder but single-channel continuous output).
- **Where to share:** share encoder and FPN up to feature maps; provide separate decoders/heads for per-task outputs. Optionally share shallow decoder layers between segmentation and depth (both dense prediction) with small task-specific final layers.

20. Gradient analysis: task A gradient 10× larger than B — problem and fix.

Problem: large gradient magnitude from A dominates parameter updates → task B underfits (imbalance). Solutions:

- **Gradient normalization (GradNorm):** rescale task losses so gradients have comparable magnitudes by learning loss weights.
 - **Per-task learning rates / loss weighting:** downweight task A's loss by a factor ~ 10 or use dynamic reweighting based on rate of loss change.
 - **Gradient clipping or rescaling methods** (e.g., normalize gradient norms per task before summing).
-

Meta-Learning Problems

21. Few-Shot scenario (5-way 1-shot, 15 query per class).

- Support set examples: $5 \times 1 = 5$.
- Query set examples: $5 \times 15 = 75$.
- Total examples per episode: $5 + 75 = 80$.

22. MAML computation (3 adaptation steps SGD).

- **Inner loop update (notation):** let ϕ be meta-init, learning rate α . Given support loss L_{support} .

$$\text{Step 1: } \theta^{(1)} = \phi - \alpha \nabla_{\phi} L_{\text{support}}(f_{\phi})$$

$$\text{Step 2: } \theta^{(2)} = \theta^{(1)} - \alpha \nabla_{\theta^{(1)}} L_{\text{support}}(f_{\theta^{(1)}})$$

$$\text{Step 3: } \theta^{(3)} = \theta^{(2)} - \alpha \nabla_{\theta^{(2)}} L_{\text{support}}(f_{\theta^{(2)}})$$

- **Meta-gradient flow:** outer loss is evaluated on query set using $\theta^{(3)}$. The gradient $\nabla_{\phi} L_{\text{query}}(f_{\theta^{(3)}})$ requires backpropagating through each inner update (chain rule) because $\theta^{(3)}$ depends on ϕ . This yields higher-order derivatives (unless first-order approximations are used).
-

PEFT / LoRA Problems

23. Parameter calculation

Given matrices:

- Dense layer: 768×3072 (call W_1)
- Attention layer: 768×768 (call W_2)

(a) Total parameters in full fine-tuning:

Compute precisely:

- $768 \times 3072 = 768 \times (3000 + 72) = 768 \cdot 3000 + 768 \cdot 72$.
 $768 \cdot 3000 = 2,304,000$.
 $768 \cdot 72 = 55,296$.
 So $768 \times 3072 = 2,359,296$.
- $768 \times 768 = 768 \times (700 + 68) = 768 \cdot 700 + 768 \cdot 68$.
 $768 \cdot 700 = 537,600$.
 $768 \cdot 68 = 52,224$.
 So $768 \times 768 = 589,824$.
- **Total** = $2,359,296 + 589,824 = 2,949,120$ parameters.

(b) Parameters for LoRA (rank $r = 16$) on both matrices

LoRA parameter count per matrix = $r(d_{\text{in}} + d_{\text{out}})$.

- For W_1 (768×3072): $r(3072 + 768) = 16 \cdot 3840 = 61,440$.
- For W_2 (768×768): $16 \cdot (768 + 768) = 16 \cdot 1536 = 24,576$.
- **Total LoRA params** = $61,440 + 24,576 = 86,016$.

(c) Percentage reduction in trainable parameters

$$\text{Trainable fraction} = \frac{86,016}{2,949,120}.$$

Compute: divide numerator and denominator by 16: $86,016/16 = 5,376$,
 $2,949,120/16 = 184,320$. So fraction = $5,376/184,320$. Evaluate:

$184,320/5,376 \approx 34.285714$, so the fraction $\approx 1/34.285714 = 0.0291667$ (exactly $7/240 \approx 0.0291667$).

- Percentage trainable $\approx 2.91667\%$.
- Percentage reduction $\approx 100\% - 2.91667\% = 97.08333\%$.

So LoRA reduces trainable parameters by about **97.08%** relative to full fine-tuning for these two matrices.

24. LoRA backpropagation gradients (derive updates for B and A)

Forward pass (per example, vector notation):

$$h = W_0 x + B A x,$$

where $x \in \mathbb{R}^{d_{\text{in}}}$, $A \in \mathbb{R}^{r \times d_{\text{in}}}$, $B \in \mathbb{R}^{d_{\text{out}} \times r}$. Let scalar loss $L(h)$. Define $g \equiv \frac{\partial L}{\partial h}$ (column vector of size d_{out}).

Compute intermediate: $s \equiv Ax$ (size r), so $BAx = Bs$.

- **Gradient w.r.t. B :** treat s as constant w.r.t. B :

$$\frac{\partial L}{\partial B} = \frac{\partial L}{\partial h} \frac{\partial h}{\partial B} = g s^\top.$$

So the update (SGD) for B with learning rate η is:

$$B \leftarrow B - \eta (g s^\top).$$

- **Gradient w.r.t. A :** use chain rule through $s = Ax$ and $h = Bs$:

$$\frac{\partial L}{\partial A} = (B^\top g) x^\top.$$

So the SGD update for A :

$$A \leftarrow A - \eta ((B^\top g) x^\top).$$

If using minibatch, g and x are averaged over batch; add usual regularization terms if present.

Section C — Comparative & Critical Thinking

25. Data efficiency: SSL vs. MTL vs. Meta-learning — which best when labeled data is extremely scarce?

- **SSL:** extremely data-efficient because it leverages massive unlabeled data to learn general representations, then fine-tunes with small labels. Best when unlabeled data is abundant.
- **MTL:** data-efficient when related labeled tasks exist (leverages labels across tasks). Requires other labeled tasks.
- **Meta-learning:** excels in few-shot adaptation when many *tasks* (not labels) are available at meta-train time; needs a task distribution.

Verdict: If you have *lots of unlabeled data*, SSL is most suitable. If you have many related labeled tasks, meta-learning or MTL can be better. For extremely scarce labels and no other labeled tasks, SSL (pretrain) + PEFT is typically best.

26. How SSL and PEFT address pretrain+finetune differently.

- **SSL:** modifies *pre-training objective* (self-supervised pretext) to learn representations across unlabeled data; then downstream step fine-tunes (or trains

head). The *base model weights are modified during pretraining*.

- **PEFT:** keeps pretraining procedure unchanged (typically supervised or SSL pretraining) but modifies *fine-tuning procedure* to only add/update small modules (adapters, LoRA) while freezing the bulk of pre-trained weights. So SSL changes the pretrain stage; PEFT changes the fine-tuning stage.

27. Trade-offs: MTL (simultaneous) vs. PEFT (sequential adapters).

- **Training cost:** MTL trains one large model jointly — can be cheaper than training many models but may be harder to converge. PEFT trains small adapters per task sequentially — cheaper per task.
- **Inference cost:** MTL inference is a single model that may predict many tasks with a shared backbone — efficient. PEFT requires loading base model + adapter per task; switching tasks might involve loading a small adapter only (memory light).
- **Storage:** MTL stores one model; PEFT stores base model once plus small adapters per task (scales well).
- **Flexibility & negative transfer:** PEFT avoids negative transfer (isolated adapters) and is safer for task addition; MTL can get transfer benefits but risks negative transfer.

Critical Analysis

28. Critique: "SSL eliminates need for labeled data entirely."

Refutation: SSL greatly reduces labeling needs but does not entirely eliminate labels because:

- Downstream tasks often require *label supervision* for fine-tuning/classifier calibration to reach top performance.
- SSL may learn representations that are good for general structure but not optimal for specific discriminative tasks without some labels.
- Some domains require fine-grained labels or rare event detection that SSL alone may not capture.
- Evaluation and validation still require labeled held-out sets.

29. When choose full fine-tuning over PEFT?

Choose full fine-tuning when:

- **Task is highly different** from pretraining distribution and needs pervasive weight updates.
- **Maximum performance** is crucial and small gains from full tuning justify cost.

- **Compute/storage resources available** (and you need only a few tasks so duplication is acceptable).
- **PEFT's expressivity limited** for the task (complex domain shift requiring full weight adaptation).

30. Debate: "Meta-learning fundamentally different from transfer learning."

Balanced position: meta-learning and transfer learning share the goal of using prior experience; differences:

- **Meta-learning** explicitly learns *how to adapt* quickly (learning algorithms, initializations, metrics) from a distribution of tasks — focuses on fast generalization to new tasks.
- **Transfer learning** usually involves training on one (or many) source tasks and transferring weights/features directly; adaptation can be simple fine-tuning.

You can *refute* strong separation: many transfer learning techniques (e.g., fine-tuning pre-trained models) are a form of learning-to-learn in the broad sense.

But *support* separation: meta-learning explicitly optimizes for adaptation speed across tasks, a different objective than standard transfer.

Section D — Advanced & Integration

31. Hybrid: SSL pretrain + PEFT adapt (proposed workflow).

- **Pretrain:** large model pretrained with SSL (contrastive or masked modeling) on abundant unlabeled data to learn a strong backbone.
- **PEFT adaptation:** for each downstream task, freeze backbone and train small LoRA adapters or adapters/prompt tokens. Optionally fine-tune a small head.
Benefits: SSL provides general representations; PEFT enables economical task specialization and multiple adapters per task for easy switching.

32. Forward vs. backward transfer

- **Definitions:**
 - **Forward transfer:** learning on previous tasks improves learning on a new task (positive transfer to future tasks).
 - **Backward transfer:** learning new tasks improves or degrades performance on previous tasks (positive = improvement, negative = forgetting).
- **Which technique is better:**
 - **Forward transfer:** meta-learning (explicitly optimizes for fast adaptation) and SSL (good features) both excel at forward transfer. Meta-learning particularly

optimized for quick few-shot forward transfer.

- **Backward transfer:** MTL (joint training) can produce positive backward transfer if tasks are jointly learned; PEFT tends to *avoid* backward transfer (adapters are isolated) and reduces catastrophic backward transfer but does not produce positive backward transfer.
- **Examples:** forward transfer matters when adapting to a new low-resource language using knowledge from high-resource ones (meta-learning/SSL helps). Backward transfer matters in continual learning where new tasks should improve older tasks (joint MTL may help if tasks are related).

33. Scalability comparison

Consider training cost, inference cost, storage, adding new tasks:

- **Separate models per task:**
 - Training cost: high (train each model separately).
 - Inference cost: small per model but if many tasks needed simultaneously, heavy.
 - Storage: large (full copy per task).
 - Adding new tasks: expensive.
- **MTL with shared backbone:**
 - Training: moderate to high (single big joint training), but more efficient than training multiple full models.
 - Inference: efficient if tasks can share backbone in a single pass.
 - Storage: one shared model.
 - Adding tasks: requires retraining or fine-tuning joint model (can be costly), risk of negative transfer.
- **PEFT with LoRA adapters:**
 - Training: cheap per task (train small adapters).
 - Inference: base model + adapter; additional compute minimal.
 - Storage: base model once + small adapters per task (very efficient).
 - Adding tasks: easy — train and store a small adapter. Best scalability for many tasks.

34. Negative transfer analysis (for MTL, SSL, Meta-learning, PEFT)

- **MTL**
 - How: tasks conflict; shared weights cause interference.
 - Mitigation: selective sharing, gradient surgery, loss weighting, task grouping.

- Susceptibility: **high** (depends on task relatedness).
- **SSL**
 - How: pretext objective misaligned with downstream tasks (learns features irrelevant or invariant to needed distinctions).
 - Mitigation: choose multiple pretext tasks, fine-tune with small labeled set, hybrid SSL + supervised signals.
 - Susceptibility: **moderate**.
- **Meta-learning**
 - How: meta-train tasks not representative of target tasks causing poor transfer; overfitting to meta-train tasks.
 - Mitigation: diversified task distribution, regularization, meta-validation on held-out tasks.
 - Susceptibility: **moderate** (depends on task distribution coverage).
- **PEFT**
 - How: limited expressivity of adapters might fail to adapt fully, but isolation prevents harming other tasks.
 - Mitigation: increase adapter rank/size, mix adapters with selective fine-tuning.
 - Susceptibility: **low** (least likely to cause negative transfer because base model frozen).

Ranking by susceptibility (most → least): MTL > SSL ≈ Meta-learning > PEFT.

35. Experiment design: SSL pretrain vs. MTL on 5 vision tasks

- **Dataset requirements:** a shared unlabeled corpus (for SSL pretraining) and a labeled multi-task dataset containing the same images annotated for 5 tasks (classification, segmentation, depth, detection, keypoint). Tasks must be related but distinct. Split tasks into train/validation/test; have held-out tasks for generalization checks.
- **Conditions:**
 - **Condition A:** SSL pretrain (large unlabeled set), then fine-tune separately on each labeled task head (or using PEFT).
 - **Condition B:** MTL train from scratch (or from supervised ImageNet init) jointly on all 5 tasks (shared backbone + heads).
 - Control for compute budget, backbone architecture, and training epochs.

- **Evaluation metrics:** task-specific: mAP for detection, IoU for segmentation, RMSE for depth, accuracy for classification, PCK for keypoints. Also measure sample efficiency — performance as labeled data is reduced (e.g., 100%, 10%, 1% labels).
 - **Statistical analysis:** run multiple random seeds (≥ 3) and compute mean $\pm$ SE. Use paired statistical tests (e.g., paired t-test or Wilcoxon) across seeds and tasks. Report effect sizes and confidence intervals. Analyze per-task gains and aggregate score (normalized).
-

Section E — Practical Application Questions (Case Studies)

36. Medical imaging (scarce labels, multiple tasks, privacy constraints)

Proposed strategy:

1. **SSL pretraining** on unlabeled medical images (privacy-preserving: federated SSL or on local institutional datasets) using masked image modeling or contrastive learning to learn robust representations.
2. **PEFT** for each task: attach LoRA adapters or small segmentation/classification heads and fine-tune adapters locally on labeled examples. Store only adapters (small) and share encrypted/aggregated updates if collaborative learning needed.
3. **Optionally MTL:** where labels exist for multiple tasks within same institution, perform local MTL for those tasks to improve shared features, combined with adapter per external deployment.

Justification: SSL reduces label need; PEFT allows small, private, per-task adapters without sharing patient data; federated SSL or private aggregation respects privacy constraints.

37. Multilingual NLP (50 languages: high/low resources)

Design:

- **Meta-learning for language adaptation:** meta-train across high-resource language tasks to learn initializations or adaptation strategy (e.g., MAML or metric-based for embeddings). This produces a multilingual meta-init that can be quickly adapted.
- **PEFT (LoRA) per language:** when adapting to a low-resource language, train a small LoRA adapter on few-shot data (or synthetic translated data) starting from the meta-init. Store one adapter per language.

- **Cross-lingual transfer:** bootstrap low-resource adapters by mixing adapters from typologically similar high-resource languages or through adapter interpolation. Benefits: meta-learning improves few-shot forward transfer; PEFT keeps adapters small (efficient to deploy for 50 languages).

38. Edge device deployment: MTL vs. PEFT (which is more suitable?)

Argument for **PEFT** as more suitable overall for mobile edge with multiple tasks and regular updates:

- **Inference efficiency:** MTL can run a single network for multiple tasks efficiently if tasks needed simultaneously. However MTL model may be large (to achieve good performance across tasks) and harder to update per task.
- **Updates & new tasks:** PEFT allows adding new tasks by deploying small adapters (download small adapter) without replacing the whole model — ideal for edge devices with limited bandwidth/storage.
- **Memory & compute:** PEFT keeps base model the same (cached on device) and loads only a small adapter; adapters add minimal compute. MTL may require a larger backbone and more memory if designed to be universal.
- **Tradeoff:** if the device must perform *many tasks simultaneously in a single run* and latency is critical, a carefully engineered MTL model might be preferable. But for flexibility, frequent updates, and storage constraints → **PEFT** is typically better.

If you want, I can (a) convert these answers into succinct exam-style bullet answers for printing, (b) produce worked derivations for any question in more detail (e.g., gradients with matrix calculus), or (c) generate sample exam solutions with step-by-step calculations and references. Which of those would you like next?